What's new in DeployHub

Everything delivered on 26 October 2026: real deployments, environment variables, fixed addresses, routing, one-click databases, public links and Forgot password

> **In short:** DeployHub now really deploys. It fetches a public repository, builds it with Docker, runs it in a locked-down container on this Mac and only calls it LIVE once the app answers. Each project gets a fixed address that survives redeploys, can receive encrypted settings such as database passwords, and can send a path like **/api** to another project, so a frontend and its backend work together without CORS. Apps get a MySQL database in one click and, if you switch it on, a public https link. Users who forget their password can now reset it with a code sent to their email.

Newest: Forgot password with an email code

The login page now has a **Forgot password?** link. It works in two steps:

Step	What happens
1. Enter your email	DeployHub emails a **6-digit code** (through Gmail). The page always says "if this email has an account, a code is on its way", so nobody can use it to find out who has an account.
2. Enter the code and a new password	If the code is right, the password is changed and you log in with the new one. Failed logins from before the reset no longer block you.

- **Safe codes:** only a scrambled (BCrypt) copy is stored. A code works **once**, for **10 minutes**, with **5 guesses**; asking again sends a new code and the old one stops working.
- **No flooding:** at most 5 codes per email address and 20 per network per hour; too many wrong codes from one network are blocked for an hour.
- **To send real emails**, put two lines in `deployhub-backend/.env`: `MAIL_USERNAME=your.address@gmail.com` and `MAIL_PASSWORD=<16-letter App password>`. The App password is made at myaccount.google.com → Security → 2-Step Verification → App passwords (not your normal Gmail password). Restart the backend afterwards.
- **Without those lines** no email is sent: the code appears in the backend's log instead (handy for testing on this Mac).
- New table `PASSWORD_RESET_CODES` (schema version V9); every request, wrong code and reset is written to the audit log.

1. Real deployments (Milestone 6)

Pressing Deploy now goes through four real steps:

Step	What happens
Get the code	A read-only, shallow `git fetch` of the branch, tag or commit from a public repository. The exact commit ID is recorded.
Build	`docker build` with a Dockerfile DeployHub writes for the app type (Java/Spring Boot, Node.js, static site, Python), or with the repository's own Dockerfile. The build command runs inside Docker, never directly on your Mac.
Run	`docker run` with limits (512 MB memory, 1 CPU, 256 processes), all extra Linux privileges removed, as an unprivileged user, reachable only from this Mac.
Health check	The app must answer HTTP on its health check path (default /). Only then is it LIVE, and the replaced version's container is stopped.

- **Your GitHub login is never used.** git runs with saved credentials switched off (macOS keychain, VS Code's helper), so a private repository fails with "Repository not found or not public" instead.
- **Nothing outside your Mac is changed:** no pushes, no webhooks, no settings on GitHub or GitLab.
- **Cancel stops a running build within about 2 seconds.** Long builds keep their claim on the job through a background heartbeat.
- **Rollbacks are exact and fast:** they redeploy the recorded commit and reuse the stored image when nothing changed. Images of the last 3 successful deployments per project are kept.
- **Clear errors:** a missing branch, a failing build command or a crashing app fail at once with the reason and the app's last output. Docker not running or network trouble is retried automatically.
- **The simulator still exists** (`deployhub.execution.mode=SIMULATED`) for the automated tests and for running without Docker.
- **New project settings:** the app's **port** (default per type: Java 8080, Node/static 3000, Python 8000) and **health check path**. The start command is optional for Dockerfile projects.

2. Fixed address per project

Each project gets one address, for example `http://localhost:19001`, at its first real deployment. It never changes: redeploys and rollbacks just switch it to the new app, without downtime.

- A small proxy container (nginx, 64 MB) holds the address and forwards to the project's live app.
- **Open app** on the project and deployment pages uses this address.
- With no live app, the address answers "No live deployment of project ..." instead of an error page.

3. Routes: frontend and backend together

A project can send a path of its address to another project, for example **/api** to your backend:

Request to the frontend's address	Goes to
`http://localhost:19002/`	the frontend project's app (your Angular files)
`http://localhost:19002/api/employees`	the backend project's app (your Express API)

- Your frontend calls `/api/...`: no backend URL in its code, and no CORS setup, because to the browser everything comes from one address.
- When the backend is redeployed, every project that routes to it follows automatically.
- Set routes on the project page (card **Routes**); they apply at once.

4. Environment variables

Settings such as `DB_HOST`, `DB_PASSWORD` or `JWT_SECRET` are given to your app when it starts (card **Environment variables** on the project page).

- **Encrypted** (AES-256-GCM) before they are stored; the key lives in your git-ignored `.env` as `DEPLOYHUB_SECRETS_KEY`. Never change or lose it.
- **Write-only:** after saving, only the name is ever shown. To change a value, type a new one.
- **Kept out of sight:** passed to Docker by name only (never on a command line) and replaced by *** if the build or the app prints them.
- They apply at the next deployment. `PORT` is set by DeployHub and can't be overridden.

5. One-click databases

On a project's page, **Add MySQL database** creates a database just for that project and fills in its settings (`DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`, `DATABASE_URL`) as encrypted environment variables.

- All projects share one MySQL 8.4 server (`deployhub-mysql`), started on first use; each project's user can only open its own database.
- The password is shown once; **Reset password** makes a new one and updates the settings.
- From this Mac you can connect at `127.0.0.1:3307` (e.g. to create tables). Details in database.pdf.

6. Public links

- Switch on **Public link** on a project and it gets an https address anyone on the internet can open (a free Cloudflare quick tunnel; no account needed). **Open app** then opens that link.
- The link works while this Mac and Docker are running, and it changes when the tunnel restarts; DeployHub checks it every 30 seconds and always shows the current one.
- Make the frontend public and keep the backend private: it is reached through the frontend's `/api` route.

7. Smaller improvements

- **Delete deployments:** failed, cancelled and replaced ones can be deleted (button on the deployment page, trash icon in the list), with their logs and history. The live one and ones in progress can't.
- **Apps can write to their own folder** (e.g. `uploads/`). Those files are lost on redeploy, as on other platforms.
- **Clearer labels:** deployments show "Docker" or "Simulated", the exact commit, and the app's address. The project button is now called **Edit project** (not to be confused with account Settings).
- **Clearer build errors:** a failed build shows the real error line, not Docker's summary or a Docker Desktop link.

8. Earlier today: Milestone 5 and fixes

Feature	In short
Live logs	Lines appear as they are written (Server-Sent Events); after a dropped connection the browser continues where it stopped. Falls back to polling.
Metrics	Queue depth, wait time, throughput, retries and durations for Prometheus, plus a "Last 60 minutes" panel on the dashboard.
Health checks	Liveness (is it running?) and readiness (database and worker OK?); graceful shutdown.
Rollback	Redeploy an earlier deployment that was live, as a new deployment.
Fixes	15 bugs found and fixed in total; see bugreport.pdf for every one.

Tests

Suite	Start of today	Now
Backend on H2 (quick, in-memory database)	112	**232**
Backend on real Oracle (Docker)	72	**136**
Real Docker: git, build, run, proxy, routing, MySQL	0	**7**
Frontend	70	**105**

Load test: 100 users at the same time

New file `load-tests/hundred-users.js` (k6). 100 virtual users each sign up, create a project and deploy it, then keep using the dashboard, projects, deployments and logs for 2 minutes. It runs against a separate test backend (`load-tests/start-test-backend.sh`, port 8081, in-memory database, simulated deployments), so your real data is never touched.

Result (26 Oct 2026, this Mac)	Value
Requests / errors	71,991 requests (433 per second), **0 errors**
Checks	**100% passed**, incl. every user sees only their own project, another user's project = 404, no login = 401
Speed (95% of requests faster than)	reading pages 1.7 ms · sign-up, create, deploy 119 ms
Deployments	all 100 queued and LIVE within 4.4 s (simulated)

Run it: `./load-tests/start-test-backend.sh`, then in a second Terminal `k6 run load-tests/hundred-users.js`. Real Docker builds were not load-tested: this Mac can build only a few apps at once.

How to deploy an app with a frontend, a backend and a database

Step	What to do
1. Database	On the backend project, click **Add MySQL database**. The DB_* settings are filled in for you.
2. Backend project	Type Node.js (or your stack). If the code is in a subfolder: build `cd backend && npm ci`, start `cd backend && npm start`. Add other environment variables (e.g. `JWT_SECRET`). Deploy.
3. Frontend project	Type Static site. In your code, call the API at `/api`. Add the route /api -> backend project. Deploy, switch on Public link if others should see it, and click Open app.

Still limited

- Apps run on this Mac: public links work only while it is on, and change when the tunnel restarts. A server in the cloud with fixed https addresses is Milestone 7.
- No backups of Oracle or MySQL yet.
- A password reset doesn't log out sessions already open on other devices (they end after 30 idle minutes). Gmail sends about 500 emails a day at most.
- Public repositories only; no build on push (webhooks would change things on GitHub).
- Files an app writes are lost on redeploy; a LIVE app isn't monitored after its health check.
- Environment variables are available when the app runs, not during its build.
- Docker has about 4 GB here, shared with Oracle: keep only a few apps live at once.

Details: README.md · decision records docs/adr/0006 to 0009 · databases: docs/database.pdf · libraries: docs/third-party-libraries.pdf · bugs: docs/bugreport.pdf

What's new: 27 October 2026

Paid deploys with Razorpay, plus load tests and a frontend setting

> **In short:** your first project is free. Deploying any other project costs **Rs. 49 per deploy**, paid on Razorpay's own payment page. The project page asks before you pay, and deploys as soon as the payment is confirmed. A deploy that fails or is cancelled gives the paid deploy back.

Newest: Paid deploys

Step	What happens
1. First deploy	The first project you deploy becomes your **free project**. Deploying it again after code changes stays free. Its page says "Free project".
2. Another project	Its page says "Rs. 49 per deploy". Pressing Deploy asks: **Pay Rs. 49 & deploy**?
3. Pay	Razorpay's window opens (UPI, cards, netbanking). Card and UPI details are typed into Razorpay's page, never into DeployHub.
4. Deploy	DeployHub checks the payment with Razorpay, adds one paid deploy, and deploys at once.

- **One payment = one deploy**, usable on any of your paid projects. **Rollbacks are free.**
- **Fair:** if the deploy fails or you cancel it, the paid deploy comes back for the next try.
- **Safe:** the server checks Razorpay's signature (made with the secret key, which never leaves the backend) and Razorpay's own record: right order, right amount, money received. Paying twice for one order or reusing a payment is impossible.
- **Closed the browser after paying?** The payment is picked up the next time the page loads.
- **Projects you already had stay free**: they were made when everything was free.
- **No PAN or KYC from your users:** that's only for you as the seller, done with Razorpay.

How to switch it on

Where	What to do
Razorpay Dashboard	Account & Settings → API Keys → generate **test** keys (rzp_test_...). Test mode works before your KYC is approved and moves no real money.
deployhub-backend/.env	Add RAZORPAY_KEY_ID=rzp_test_... and RAZORPAY_KEY_SECRET=... then restart the backend. Never put the secret in the frontend or on GitHub.
Try it	Deploy a second project; in Razorpay choose Netbanking, any bank, Success.
Price	deployhub.billing.price-per-deploy in application.properties, in paise (4900 = Rs. 49).

Without keys, your free project still works, and paid deploys explain that payments aren't set up yet. Not done yet: refunds of money, invoices/GST, and Razorpay webhooks.

Also new

- **Frontend proxy port:** if the backend runs on another port, start the frontend with API_PORT=8081 npm start.
- **Live-link load test:** k6 run load-tests/live-links.js (10 users; -e USERS=100 for 100). Each user deploys for real and must get a working link. Last full run: 86 of 90 got a working link; the other 4 hit BUG-16 (see the bug report).
- **Cleanup:** k6 run load-tests/cleanup-live-links.js removes everything a stopped or KEEP=1 load test left behind.

What's new: App folder

27 October 2026 · for repositories with backend/ and frontend/ folders

In short: the project form has a new box, **App folder**. Type the folder that has your app's files, e.g. **backend** or **frontend**. DeployHub goes into that folder and runs the build and start commands there, so you write plain commands like **npm ci** and **npm start**, without "cd backend &&".

Before and after

	Before	Now
App folder	(not possible)	backend
Build command	cd backend && npm ci	npm ci
Start command	cd backend && npm run db:setup && npm start	npm run db:setup && npm start

- **Empty box** = the repository's main folder, exactly as before. Existing projects keep working.
- **Wrong folder name?** The deployment stops at once with "The folder 'backend' isn't in the repository" instead of a confusing build error.
- **Safe:** only folders inside the repository are accepted (no "..", no paths starting with /).
- **Dockerfile projects:** the Dockerfile inside the folder is used, with that folder as the build context.
- **Checked for real:** your Employee Management frontend deployed with App folder **frontend**, build npm ci && npx ng build, and went LIVE (health check HTTP 200).

What's new: 2 free projects and a payment page

27 October 2026 · changes to paid deploys

> **In short:** every account now gets **2 free projects** instead of 1, so an app with a backend and a frontend (like your Employee Management) costs nothing. Paying for a deploy now happens on its own page.

2 free projects

- The **first two projects you deploy** are free, and so are all their redeploys. The third project costs Rs. 49 per deploy.
- **Existing accounts get 2 as well**, including yours: you have 2 free projects available now.
- Deleting a free project doesn't give it back (so nobody can delete and recreate to stay free).
- The project page says how many are left, e.g. "Free: your first deploy makes this one of your free projects (2 of 2 left)".
- The number is a setting: deployhub.billing.free-projects=2 in application.properties.

Payment page

Step	What happens
Deploy a paid project	DeployHub opens /app/projects/payment?project=<id> instead of a popup.
The page shows	The project, the price, how many paid deploys you have left, and Pay Rs. 49 & deploy.
After paying	Razorpay confirms, the deploy starts, and the deployment's page opens.
Paid deploy left?	The page shows Deploy now and uses it, without a new payment.

Test mode tip: in the Razorpay window choose Netbanking, any bank, then Success. On a computer Razorpay shows UPI only as a QR code.

What's new: the right commands for each type

27 October 2026 · project form

In short: picking an **Application type** now always fills in that type's commands, on New and Edit, and switching type switches them. Before, the commands stayed as they were once you had typed in a box, and the Edit page never filled them in.

Commands per type

Type	Build command	Start command
Node.js	npm install	npm start
Static site	npm install && npm run build	serves the first build folder that exists: dist/*/browser (Angular), dist (Vite, Vue), build (React), out (Next export)
Java / Spring Boot	./mvnw -B -DskipTests package	java -jar target/*.jar
Python	pip install --no-cache-dir -r requirements.txt	gunicorn -b 0.0.0.0:$PORT app:app
Dockerfile	(none)	(the Dockerfile's own CMD)

- **Your own commands are kept.** If you wrote a command yourself, picking a type doesn't overwrite it; a "Use the suggested ... commands" button switches with one click.
- Node.js now uses npm install (it also works when the repository has no package-lock.json).
- Apps with an extra step still need it, e.g. your Employee Management backend: npm run db:setup && npm start.

What's new: build errors that say what to do

27 October 2026 · deployment messages

In short: when a build fails because of a common command mistake, the failure message now adds what to change, in plain words.

The build says	DeployHub now adds
npm error Missing script: "build"	Your package.json has no "build" script, so "npm run build" can't run. Remove it from the command (a Node.js backend usually needs only npm install or npm ci), or check the App folder points at the right package.json.
npm ci can only install with an existing package-lock.json	The repository has no package-lock.json there: use npm install instead of npm ci.

Found while deploying Employee Management: the backend project had the frontend's build command (npm ci && npm run build).

What's new: Detect from repository

27 October 2026 · the right commands, automatically

In short: DeployHub now reads your repository and fills in the application type and the commands that fit **your** project, so users no longer need to know them. On a new project it starts by itself as soon as the repository URL is filled in; the **Detect from repository** button runs it again.

Your Employee Management repository

App folder	Detected	Commands
(empty)	No app here	Offers the folders that have one: backend, frontend (one click picks one)
backend	Node.js	Build: npm ci · Start: npm run db:setup && npm start
frontend	Static site (Angular)	Build: npm ci && npm run build · Start: serve dist/ems-frontend/browser (read from angular.json)

What it looks at

- **package.json**: Angular, Vite, React, Next or a Node backend; "npm run build" only if a build script exists; database scripts such as db:setup run before the app starts.
- **Lock files**: npm ci, yarn or pnpm when there is one, otherwise npm install.
- **Java and Python**: Maven or Gradle (warns when the wrapper is missing), Django, FastAPI, Flask (warns when gunicorn or uvicorn is missing). **Dockerfile**: the Dockerfile type.
- It explains what it found in plain words, and you can still change every command before saving.
- Safe and quick: it reads the repository with git, read-only, only the file list plus the few files it needs (1-3 seconds). Works for GitHub, GitLab and Bitbucket.

What's new: What this app needs

27 October 2026 · environment variables filled in for you

> **In short:** users no longer need to know which environment variables an app needs or how to make a secret key. The project page reads the repository, lists what the app needs, and **Fill in automatically** sets everything DeployHub can: it generates secrets and uses the example values.

How each variable is handled

Kind	Example	DeployHub
App's own secret	JWT_SECRET, DEPLOYHUB_SECRETS_KEY	Generates a strong random value (nobody needs to see it)
Setting with example	JWT_EXPIRES_IN=24h, NODE_ENV	Uses the example value (production for NODE_ENV)
Database	DB_HOST, DATABASE_URL	Points to Add MySQL database, which sets them
Set by DeployHub	PORT	Nothing to do
Key from another service	STRIPE_SECRET_KEY, SMTP_PASSWORD	Explains that only you can get it
Laptop-only example	CORS_ORIGIN=http://localhost:4200	Warns: usually leave it out

- **Where it looks:** deployhub.json (if the app's author wrote one), .env.example, and the code itself (process.env, Spring ${...}, Python os.environ).
- **Deploy checks first:** missing settings? "Fill in & deploy", or it says what only you can add.
- **If the app still crashes**, the message names the missing variable, e.g. "The app seems to need JWT_SECRET, which isn't set".
- **Your Employee Management backend:** JWT_SECRET generated, JWT_EXPIRES_IN=24h and NODE_ENV=production filled in, the database offered, PORT skipped, CORS_ORIGIN flagged.
- **DeployHub itself:** the new deployhub-backend/deployhub.json makes a demo copy one click: SPRING_PROFILES_ACTIVE=local, DEPLOYHUB_EXECUTION_MODE=SIMULATED and a generated DEPLOYHUB_SECRETS_KEY. Commit and push it to GitHub so DeployHub can read it.